FINAL MASTER BLUEPRINT — COMPLETE SPRING BOOT DATABASE DESIGN

This is the finalized source of truth for your project.

Your platform is a hybrid of:

- LinkedIn
- Fiverr
- Indeed
- Bdjobs

The system includes:

- Job portal
- Freelancer marketplace
- CV builder
- Wallet/payment system
- Chat/messaging system
- Dispute management
- Admin management

The architecture is intentionally:

- beginner-friendly
- monolithic
- scalable
- Bootstrap-based
- Angular + Spring Boot + MySQL based

You are intentionally avoiding:

- microservices
- Kubernetes
- enterprise complexity
- event-driven architecture

That is the correct decision for your project.

MAIN USER ROLES

UserRole Enum

```
ADMIN
USER
COMPANY
```

MAIN SYSTEM MODULES

```
Authentication System
User Profile & CV System
Company & Hiring System
Job System
Gig/Freelancer System
Wallet & Payment System
Messaging System
Dispute System
Notification System
Location System
Category & Skill System
```

IMPORTANT ENUMS

ApplicationStatus

```
APPLIED
SHORTLISTED
HIRED
REJECTED
WITHDRAWN
```

Meaning:

Enum	Meaning
APPLIED	User applied
SHORTLISTED	Company selected candidate
HIRED	User hired
REJECTED	Company rejected
WITHDRAWN	User withdrew application

GigOrderStatus

```
INITIATED
QUOTE_SENT
QUOTE_REJECTED
ACTIVE
DELIVERED
BUYER_REJECTED
DISPUTED
COMPLETED
REFUNDED
CLOSED
```

Meaning:

Enum	Meaning
INITIATED	Buyer started order
QUOTE_SENT	Freelancer sent price
QUOTE_REJECTED	Buyer rejected quote
ACTIVE	Buyer accepted quote and payment locked
DELIVERED	Freelancer delivered work
BUYER_REJECTED	Buyer rejected delivery
DISPUTED	Dispute opened
COMPLETED	Buyer accepted work
REFUNDED	Buyer refunded
CLOSED	Order fully closed

TransactionType

```
DEPOSIT
PAYMENT
REFUND
WITHDRAWAL
COMMISSION
RELEASE
```

NotificationType

```
MESSAGE
JOB
GIG
PAYMENT
```

```
DISPUTE
SYSTEM
```

ChatType

```
JOB
GIG
DISPUTE
```

DisputeResolution

```
PENDING
PAYMENT_RELEASED
REFUND_ISSUED
```

---

1. USER ENTITY

Java Class

```
User
```

Purpose

Stores:

- login information
- authentication data
- account status
- system role

Fields

Field	Type	Purpose
id	Long	Primary key
firstName	String	User first name

Field	Type	Purpose
lastName	String	User last name
email	String	Unique login email
password	String	Encrypted password
phone	String	Contact number
profilePicture	String	Profile image URL
role	UserRole	ADMIN / USER / COMPANY
isVerified	Boolean	Email verified or not
isActive	Boolean	Account active or inactive
isSuspended	Boolean	Suspended by admin
createdAt	LocalDateTime	Account creation time
updatedAt	LocalDateTime	Last update time

Relationships

```
@OneToOne(mappedBy = "user")
private UserProfile userProfile;

@OneToOne(mappedBy = "user")
private CompanyProfile companyProfile;

@OneToOne(mappedBy = "user")
private Wallet wallet;

@OneToMany(mappedBy = "user")
private List<JobApplication> jobApplications;

@OneToMany(mappedBy = "seller")
private List<Gig> gigs;

@OneToMany(mappedBy = "buyer")
private List<GigOrder> purchasedOrders;

@OneToMany(mappedBy = "sender")
private List<Message> sentMessages;

@OneToMany(mappedBy = "receiver")
private List<Notification> notifications;

@OneToMany(mappedBy = "user")
private List<SavedJob> savedJobs;

@OneToMany(mappedBy = "user")
private List<SavedGig> savedGigs;

@OneToMany(mappedBy = "raisedBy")
private List<Dispute> raisedDisputes;
```

```
@OneToMany(mappedBy = "resolvedByAdmin")
private List<Dispute> resolvedDisputes;
```

---

2. USERPROFILE ENTITY

Java Class

`UserProfile`

Purpose

Stores:

- CV information
- freelancer information
- professional information

Fields

Field	Type
id	Long
headline	String
bio	String
professionalSummary	String
dateOfBirth	LocalDate
gender	String
website	String
githubUrl	String
linkedinUrl	String
expectedSalary	BigDecimal
freelancerTitle	String
createdAt	LocalDateTime
updatedAt	LocalDateTime

Relationships

```
@OneToOne
@JoinColumn(name = "user_id")
private User user;

@ManyToOne
@JoinColumn(name = "present_address_id")
private Address presentAddress;

@ManyToOne
@JoinColumn(name = "permanent_address_id")
private Address permanentAddress;

@OneToMany(mappedBy = "userProfile")
private List<Education> educations;

@OneToMany(mappedBy = "userProfile")
private List<Experience> experiences;

@OneToMany(mappedBy = "userProfile")
private List<Training> trainings;

@OneToMany(mappedBy = "userProfile")
private List<Portfolio> portfolios;

@OneToMany(mappedBy = "userProfile")
private List<Reference> references;

@OneToMany(mappedBy = "userProfile")
private List<Extracurricular> extracurriculars;

@OneToMany(mappedBy = "userProfile")
private List<UserSkill> skills;

@OneToMany(mappedBy = "userProfile")
private List<UserLanguage> languages;

@OneToMany(mappedBy = "userProfile")
private List<ResumeSnapshot> resumes;
```

IMPORTANT BUSINESS RULE

Before applying for a job:

```
User must:
- complete UserProfile
- generate ResumeSnapshot
```

This validation happens in service layer.

3. COMPANYPROFILE ENTITY

Java Class

`CompanyProfile`

Fields

Field	Type
id	Long
companyName	String
companyLogo	String
companyDescription	String
website	String
industry	String
companySize	String
foundedYear	Integer
tradeLicenseNumber	String
createdAt	LocalDateTime
updatedAt	LocalDateTime

Relationships

```
@OneToOne
@JoinColumn(name = "user_id")
private User user;

@ManyToOne
@JoinColumn(name = "address_id")
private Address address;

@OneToMany(mappedBy = "company")
private List<Job> jobs;
```

LOCATION SYSTEM

Country

Field	Type
id	Long
name	String

Division

Field	Type
id	Long
name	String

Relationships:

```
@ManyToOne
private Country country;
```

District

Field	Type
id	Long
name	String

Relationships:

```
@ManyToOne
private Division division;
```

PoliceStation

Field	Type
id	Long
name	String

Relationships:

```
@ManyToOne
private District district;
```

Address

Field	Type
id	Long
addressLine	String
postalCode	String

Relationships:

```
@ManyToOne
private Country country;

@ManyToOne
private Division division;

@ManyToOne
private District district;

@ManyToOne
private PoliceStation policeStation;
```

IMPORTANT ADDRESS RULE

Address rows should NEVER be reused.

Always create a new Address row.

This prevents accidental address overwrite.

CATEGORY ENTITY

Java Class

```
Category
```

Fields

Field	Type
id	Long
name	String
description	String

Relationships

```
@OneToMany(mappedBy = "category")
private List<Job> jobs;

@OneToMany(mappedBy = "category")
private List<Gig> gigs;

@OneToMany(mappedBy = "category")
private List<Skill> skills;
```

SKILL ENTITY

Java Class

```
Skill
```

Fields

Field	Type
id	Long
name	String

Relationships

```
@ManyToOne
private Category category;
```

USERSKILL ENTITY

Java Class

`UserSkill`

Fields

Field	Type
id	Long
proficiencyLevel	String
yearsOfExperience	Integer

Relationships

```
@ManyToOne
private UserProfile userProfile;

@ManyToOne
private Skill skill;
```

LANGUAGE ENTITY

Java Class

`Language`

Fields

Field	Type
id	Long
name	String

USERLANGUAGE ENTITY

Java Class

`UserLanguage`

Fields

Field	Type
id	Long
proficiency	String

Relationships

```
@ManyToOne
private UserProfile userProfile;

@ManyToOne
private Language language;
```

EDUCATION ENTITY

Java Class

`Education`

Fields

Field	Type
id	Long
degree	String
institution	String
fieldOfStudy	String
result	String

Field	Type
startDate	LocalDate
endDate	LocalDate
currentlyStudying	Boolean

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

EXPERIENCE ENTITY

Java Class

```
Experience
```

Fields

Field	Type
id	Long
companyName	String
position	String
responsibilities	String
startDate	LocalDate
endDate	LocalDate
currentlyWorking	Boolean

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

TRAINING ENTITY

Java Class

`Training`

Fields

Field	Type
id	Long
title	String
institution	String
duration	String
completionYear	Integer

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

PORTFOLIO ENTITY

Java Class

`Portfolio`

Fields

Field	Type
id	Long
title	String
description	String
projectUrl	String
imageUrl	String

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

REFERENCE ENTITY

Java Class

```
Reference
```

Fields

Field	Type
id	Long
name	String
designation	String
organization	String
phone	String
email	String
relation	String

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

EXTRACURRICULAR ENTITY

Java Class

```
Extracurricular
```

Fields

Field	Type
id	Long
activityName	String
description	String

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

RESUMESNAPSHOT ENTITY

Java Class

```
ResumeSnapshot
```

Fields

Field	Type
id	Long
title	String
generatedPdfUrl	String
generatedHtml	String(TEXT)
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private UserProfile userProfile;
```

JOB ENTITY

Java Class

`Job`

Fields

Field	Type
id	Long
title	String
description	String(TEXT)
requirements	String(TEXT)
salaryMin	BigDecimal
salaryMax	BigDecimal
jobType	String
experienceLevel	String
deadline	LocalDate
isActive	Boolean
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private CompanyProfile company;

@ManyToOne
private Category category;

@OneToMany(mappedBy = "job")
private List<JobApplication> applications;
```

JOBAPPLICATION ENTITY

Java Class

`JobApplication`

Fields

Field	Type
id	Long
coverLetter	String(TEXT)
status	ApplicationStatus
appliedAt	LocalDateTime

Relationships

```
@ManyToOne
private User user;

@ManyToOne
private Job job;

@ManyToOne
private ResumeSnapshot resumeSnapshot;

@OneToOne
@JoinColumn(name = "chat_room_id", nullable = true)
private ChatRoom chatRoom;
```

GIG ENTITY

Java Class

```
Gig
```

Fields

Field	Type
id	Long
title	String
description	String(TEXT)
startingPrice	BigDecimal
deliveryDays	Integer
imageUrl	String

Field	Type
isActive	Boolean
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private User seller;

@ManyToOne
private Category category;

@OneToMany(mappedBy = "gig")
private List<GigOrder> orders;

@OneToMany(mappedBy = "gig")
private List<Review> reviews;
```

GIGORDER ENTITY

Java Class

```
GigOrder
```

Fields

Field	Type
id	Long
quotedPrice	BigDecimal
agreedPrice	BigDecimal
finalPrice	BigDecimal
requirements	String(TEXT)
deliveryMessage	String(TEXT)
deliveryFileUrl	String
status	GigOrderStatus
disputeDeadline	LocalDateTime
paymentLocked	Boolean
adminResolved	Boolean

Field	Type
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private Gig gig;

@ManyToOne
private User buyer;

@ManyToOne
private User freelancer;

@OneToOne(mappedBy = "gigOrder")
private Dispute dispute;

@OneToOne
@JoinColumn(name = "chat_room_id")
private ChatRoom chatRoom;
```

REVIEW ENTITY

Java Class

```
Review
```

Fields

Field	Type
id	Long
rating	Integer
comment	String
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private User reviewer;
```

```
@ManyToOne
private Gig gig;

@ManyToOne
private GigOrder order;
```

DISPUTE ENTITY

Java Class

`Dispute`

Fields

Field	Type
id	Long
reason	String(TEXT)
adminDecision	String
resolution	DisputeResolution
createdAt	LocalDateTime
resolvedAt	LocalDateTime

Relationships

```
@OneToOne
@JoinColumn(name = "gig_order_id")
private GigOrder gigOrder;

@ManyToOne
@JoinColumn(name = "raised_by_user_id")
private User raisedBy;

@ManyToOne
@JoinColumn(name = "resolved_by_admin_id")
private User resolvedByAdmin;
```

CHATROOM ENTITY

Java Class

`ChatRoom`

Fields

Field	Type
id	Long
chatType	ChatType
isClosed	Boolean
createdAt	LocalDateTime

Relationships

```
@OneToOne(mappedBy = "chatRoom")
private JobApplication jobApplication;

@OneToOne(mappedBy = "chatRoom")
private GigOrder gigOrder;

@ManyToOne
@JoinColumn(name = "participant_one_id")
private User participantOne;

@ManyToOne
@JoinColumn(name = "participant_two_id")
private User participantTwo;

@ManyToOne
@JoinColumn(name = "admin_participant_id")
private User adminParticipant;

@OneToMany(mappedBy = "chatRoom")
private List<Message> messages;
```

MESSAGE ENTITY

Java Class

`Message`

Fields

Field	Type
id	Long
messageText	String(TEXT)
isRead	Boolean
sentAt	LocalDateTime

Relationships

```
@ManyToOne
private ChatRoom chatRoom;

@ManyToOne
private User sender;
```

FILEATTACHMENT ENTITY

Java Class

```
FileAttachment
```

Fields

Field	Type
id	Long
fileName	String
fileUrl	String
uploadedAt	LocalDateTime

Relationships

```
@ManyToOne
private Message message;
```

WALLET ENTITY

Java Class

`Wallet`

Fields

Field	Type
id	Long
balance	BigDecimal
frozenBalance	BigDecimal
createdAt	LocalDateTime

IMPORTANT WALLET LOGIC

```
balance = total money

frozenBalance = locked money during active/disputed orders

availableBalance = balance - frozenBalance
```

Relationships

```
@OneToOne
@JoinColumn(name = "user_id")
private User user;

@OneToMany(mappedBy = "wallet")
private List<Transaction> transactions;
```

TRANSACTION ENTITY

Java Class

`Transaction`

Fields

Field	Type
id	Long
amount	BigDecimal
transactionType	TransactionType
sslTransactionId	String
referenceId	Long
referenceType	String
description	String
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private Wallet wallet;
```

SAVEDJOB ENTITY

Java Class

```
SavedJob
```

Fields

Field	Type
id	Long
savedAt	LocalDateTime

Relationships

```
@ManyToOne
private User user;

@ManyToOne
private Job job;
```

SAVEDGIG ENTITY

Java Class

`SavedGig`

Fields

Field	Type
id	Long
savedAt	LocalDateTime

Relationships

```
@ManyToOne
private User user;

@ManyToOne
private Gig gig;
```

NOTIFICATION ENTITY

Java Class

`Notification`

Fields

Field	Type
id	Long
title	String
message	String
notificationType	NotificationType
referenceId	Long
referenceType	String
isRead	Boolean
createdAt	LocalDateTime

Relationships

```
@ManyToOne
private User receiver;
```

FINAL DATABASE FLOW SUMMARY

Authentication Flow

```
User
↓
JWT Authentication
↓
Role-based authorization
```

CV Builder Flow

```
User
↓
UserProfile
↓
Education / Experience / Skills / Portfolio
↓
ResumeSnapshot
↓
PDF Generation
```

Job Flow

```
CompanyProfile
↓
Job
↓
JobApplication
↓
ChatRoom
↓
Hiring
```

Gig Flow

```
Buyer starts order
↓
Chat opens
↓
Freelancer sends quote
↓
Buyer accepts/rejects
↓
Payment frozen
↓
Freelancer delivers work
↓
Buyer accepts/rejects
↓
Dispute possible
↓
Admin joins dispute chat
↓
Refund OR release payment
```

Wallet Flow

```
Wallet
↓
Transaction
↓
SSLCommerz
```

Messaging Flow

```
ChatRoom
↓
Message
↓
FileAttachment
```

FINAL DEVELOPMENT ORDER

Build in this order:

1. Location system
2. Authentication
3. UserProfile system
4. Company system
5. Job system
6. Gig system
7. Wallet system
8. Chat system
9. Dispute system
10. Notification system
11. Resume PDF generation
12. SSLCommerz integration